

The Python **re** Module

A practical guide to regular expressions, with commented, runnable examples

Python's built-in **re** module implements **regular expressions** (regex) — a mini-language for describing patterns in text. It lets you search, match, split, and replace strings based on flexible patterns instead of exact substrings. Import it with `import re`.

Regex is extremely powerful for text processing, but it is also easy to overuse. A good rule of thumb: if a simple `str.split()`, `in`, or `str.replace()` can do the job, prefer that — reach for `re` when the pattern has real variability (optional parts, repetition, alternatives).

Where it's used

- Validating input formats: emails, phone numbers, postal codes
- Extracting structured data from log files or scraped text
- Find-and-replace operations that go beyond exact strings
- Tokenizing or splitting text on complex delimiters

■ Regex is **greedy** and **case-sensitive** by default, and patterns can become hard to read quickly. Add comments, use raw strings (`r"..."`), and test patterns incrementally.

1. Core Search Functions

re.search(pattern, string)

Scans the whole string and returns a Match object for the **first** location where the pattern matches, or None if there's no match.

```
import re

text = "My phone number is 555-1234."

# r"..." is a RAW string -> backslashes are treated literally,
# which regex needs a lot of (e.g. \d for "digit")
match = re.search(r"\d{3}-\d{4}", text)

if match:
    print(match.group())    # "555-1234"
    print(match.start())    # index where the match begins
    print(match.end())      # index where the match ends
```

re.match(pattern, string)

Like search(), but only checks for a match at the very **start** of the string.

```
import re

print(re.match(r"\d+", "123abc"))    # matches: starts with digits
print(re.match(r"\d+", "abc123"))    # None: doesn't start with a digit
print(re.search(r"\d+", "abc123"))   # matches: search() looks anywhere
```

re.fullmatch(pattern, string)

Requires the pattern to match the **entire** string, not just part of it. Useful for strict validation.

```
import re

# Validate that a string is ENTIRELY 4 digits, nothing more
print(bool(re.fullmatch(r"\d{4}", "1234")))    # True
print(bool(re.fullmatch(r"\d{4}", "1234x")))   # False, extra character
```

re.findall(pattern, string)

Returns a **list of all** non-overlapping matches (as strings, or as tuples if the pattern has multiple groups).

```
import re

text = "Call 555-1234 or 555-5678 for support."

numbers = re.findall(r"\d{3}-\d{4}", text)
print(numbers)    # ['555-1234', '555-5678']
```

re.finditer(pattern, string)

Like findall(), but returns an **iterator of Match objects** instead of plain strings — useful when you also need positions or groups for each match.

```
import re

text = "Call 555-1234 or 555-5678 for support."
```

```
for m in re.finditer(r"\d{3}-\d{4}", text):  
    print(f"Found {m.group()!r} at position {m.start()}")
```

2. Modifying and Splitting Strings

`re.sub(pattern, replacement, string)`

Replaces every match of `pattern` with `replacement`, returning a new string.

```
import re

text = "Contact: 555-1234, 555-5678"

# Replace every phone number with a placeholder
masked = re.sub(r"\d{3}-\d{4}", "[REDACTED]", text)
print(masked) # "Contact: [REDACTED], [REDACTED]"

# The 'count' argument limits how many matches get replaced
first_only = re.sub(r"\d{3}-\d{4}", "[REDACTED]", text, count=1)
print(first_only) # "Contact: [REDACTED], 555-5678"
```

`re.split(pattern, string)`

Splits a string wherever the `pattern` matches, returning a list of pieces — like `str.split()` but with a regex delimiter.

```
import re

text = "apple, banana; cherry ,date"

# Split on a comma or semicolon, plus any surrounding whitespace
parts = re.split(r"\s*[;,]\s*", text)
print(parts) # ['apple', 'banana', 'cherry', 'date']
```

3. Capturing Groups

Parentheses `()` in a pattern create a **group**, letting you pull out specific pieces of a match rather than the whole thing.

```
import re

log_line = "2024-03-15 ERROR Connection timed out"

pattern = r"(\d{4}-\d{2}-\d{2}) (\w+) (.+)"
match = re.search(pattern, log_line)

if match:
    date, level, message = match.groups()
    print(date)      # "2024-03-15"
    print(level)     # "ERROR"
    print(message)   # "Connection timed out"
```

Named groups

Use `(?P<name>...)` to give a group a name instead of relying on position, which makes code much easier to read.

```
import re

pattern = r"(?P<date>\d{4}-\d{2}-\d{2}) (?P<level>\w+) (?P<message>.+)"
```

```
match = re.search(pattern, "2024-03-15 ERROR Connection timed out")

if match:
    data = match.groupdict()
    print(data)
    # {'date': '2024-03-15', 'level': 'ERROR', 'message': 'Connection timed out'}
    print(match.group("level")) # "ERROR"
```

4. Pattern Syntax Cheat Sheet

Token	Meaning
.	Any character except newline
\d / \D	Digit / non-digit
\w / \W	Word character (letter, digit, _) / non-word
\s / \S	Whitespace / non-whitespace
^ / \$	Start / end of string (or line, with re.MULTILINE)
*	0 or more of the previous token
+	1 or more of the previous token
?	0 or 1 of the previous token (optional)
{n} / {n,m}	Exactly n times / between n and m times
[...]	Character class, e.g. [aeiou]
[^...]	Negated character class, e.g. [^0-9]
(...)	Capturing group
(?:...)	Non-capturing group
(?P<name>...)	Named capturing group
a b	Alternation: matches a or b
\b	Word boundary

Useful flags

Flags change how a pattern is interpreted; pass them as the third argument, e.g. `re.search(pattern, text, re.IGNORECASE)`.

Flag	Effect
<code>re.IGNORECASE</code> (<code>re.I</code>)	Case-insensitive matching
<code>re.MULTILINE</code> (<code>re.M</code>)	<code>^</code> and <code>\$</code> match at each line, not just string start/end
<code>re.DOTALL</code> (<code>re.S</code>)	<code>'.'</code> also matches newline characters
<code>re.VERBOSE</code> (<code>re.X</code>)	Allows whitespace/comments in the pattern for readability

5. Compiling Patterns for Reuse

`re.compile()` pre-compiles a pattern into a reusable `Pattern` object. This is faster when the same pattern is used many times (e.g. inside a loop), and keeps code cleaner.

```
import re

# Compile once, use many times
phone_pattern = re.compile(r"\d{3}-\d{4}")

lines = [
    "Call 555-1234 for sales.",
    "Support line: 555-5678.",
    "No number here.",
]

for line in lines:
    match = phone_pattern.search(line)
    if match:
        print(f"Found: {match.group()}")
    else:
        print("No phone number found.")
```

■ If you use a pattern only once, `re.search(pattern, text)` is fine — Python internally caches recently used patterns anyway. Compile explicitly when the pattern is reused often or when you want a clear, named reference to it.

6. Worked Example: Simple Email Extractor

A small, fully commented script that ties several pieces together to pull valid-looking emails out of a block of text.

```
import re

TEXT = """
Contact us at support@example.com or sales@example.co.uk.
Invalid entries: not-an-email, missing@dot, @nouser.com
Backup contact: youssef.dev@studentmail.org
"""

# Pattern broken down:
# [\w.+-]+      one or more word chars / dots / plus / hyphen (the user part)
# @            literal "@"
# [\w-]+       the domain name
# (\.[\w-]+)+  one or more ".something" parts (handles .co.uk too)
EMAIL_PATTERN = re.compile(r"[\w.+-]+@[ \w-]+(?:\.[\w-]+)+")

def extract_emails(text):
    """Return a list of unique, valid-looking email addresses found in text."""
    found = EMAIL_PATTERN.findall(text)
    # Use a dict instead of a set to preserve first-seen order (Python 3.7+)
    unique_emails = list(dict.fromkeys(found))
    return unique_emails

emails = extract_emails(TEXT)
```

```
for email in emails:
    print(email)

# Expected output:
# support@example.com
# sales@example.co.uk
# youssef.dev@studentmail.org
```

7. Function Reference

Function	Returns	Description
<code>re.search()</code>	<i>Match or None</i>	First match anywhere in the string
<code>re.match()</code>	<i>Match or None</i>	Match only at the start of the string
<code>re.fullmatch()</code>	<i>Match or None</i>	Match against the ENTIRE string
<code>re.findall()</code>	<i>list</i>	All matches as strings/tuples
<code>re.finditer()</code>	<i>iterator</i>	All matches as Match objects
<code>re.sub()</code>	<i>str</i>	String with matches replaced
<code>re.split()</code>	<i>list</i>	String split wherever pattern matches
<code>re.compile()</code>	<i>Pattern</i>	Pre-compiled, reusable pattern object